Setup modding

Setup modding involves creating a "savefile" for the game to open in order to play the game. This is achieved by using a set of managers and functions inside of them which are similar, but not exactly the same, as the ones you might find in Save-game editing.

Please help with verifying or updating older sections of this article.
At least some were last verified for version pre-release.

İçindekiler

File structure

Depending on the setup files, setup files are located in specific subfolders of the `setup` folder. Most setup files are in `setup/start`, but there are also specific folders for country definitions and templates under `setup/countries` and `setup/templates` respectively.
Examples include:

```
/Europa Universalis V/game/in_game/setup/countries/anatolia.txt
/Europa Universalis V/game/main_menu/setup/templates/amerindian_monarchy.txt
/Europa Universalis V/game/main_menu/setup/start/02_core.txt
```

Technical details

Most managers in the `setup/start` folder work in an additive fashion. This makes the addition to vanilla setup relatively easy, but makes it so that replacing entire files is required in order to remove certain entries.

Encoding

Load order

Some setup components need to have their object created in specific order.

- Dynasties used by characters in `character_db` need to be created in `dynasty_manager` first
- Characters referenced as spouses/fathers/mothers need to be created before their usage.

Start setup

The following subcategories outline different components of `setup/start`, what they do and allow.

Institution manager

`institution_manager` is used to define which institutions start active in the setup and what their origins are:

```
institution_manager = {
	institutions = { # this entry is required
		feudalism = { # institution key for which we define starting behavior
			active = no		# Is the institution active? Can it spread? Can it be embraced?

			birth_place = krakow	# the origin of the institution. Guarantees the location to actually have it and gives it the static modifier respective for its origin
		}
	}
}
```

Religion manager

`religion_manager` is used to create setup for religions and religious schools. Religious School opinion setup looks as the following:

```
religion_manager = {
	hanafi_school = {
		relation = {
			maturidi_school = kindred	# available: enemy, negative, neutral (default relation between schools), positive, kindred
			maliki_school = kindred
			zaidi_school = kindred
		}
	}
	hanbali_school = {	# Shortened notation- equivalent to the above
		athari_school = kindred
		mutazili_school = enemy
	}
}
```

Religion keys inserted into `religion_manager` are used to define religion specifics. Here, the religion can be given timed modifiers. Example:

```
                }
                saint = {
                        character = pol_saint_wojciech # the saint character
                        country = POL # the country the saint belongs to
                }
        }
}
```

Market manager

`market_manager` is used solely to declare which locations should start with a market center. There is no other special functionality here.

```
market_manager = {
        add_market = lubeck             # adds a market center in lubeck
}
```

Dynasty manager

`dynasty_manager` is used to declare dynasties that can be later used in character definitions.

```
dynasty_manager = {
        example_dynasty = {
                name = { name = bjalbo_atten_dynasty }  #the localizable string must be stated on the right
side of name =
                dynasty_name_type = location            #Dynasty type, accepted options: location,
location_ancient, patronym, descendant
                home = linkoping                        #home of the dynasty - location

                male_names = {                  #list of localizable strings for male names
                        name_birger name_eric name_magnus name_waldemar
                }
                female_names = {                #list of localizable strings for female names
                        name_catherine name_christine name_ingeborg name_rikissa
                }
        }
}
```

Character DB

`character_db` is used to create characters for the game start as well as their ancestors. As some of the characters will require a dynasty, it is necessary for that dynasty to be created earlier in the dynasty_manager. Characters in the character db must be created according to the order in which they are needed, that is; one cannot reference a character as a father/mother if they have not been created prior.

```
character_db = {
        …


        example_character = {   #can be later referred in script with character data scope
                first_name = { name = name_example }            # Localizable key must be inserted inside
                nickname = { name = example_nickname }          # Localizable key must be inserted inside
                last_name = { name = name_example_last_name }
                culture = swedish                               # Key referring to the culture of the
    racter
                religion = catholic                             # Key referring to the religion of the
character
```

```
                death_date = 1200.10.11                 # Death Date
                female = yes                            # Is the character female?
                birth = vadstena                        # Location of Birth
                ruler_trait = conqueror                 # Has ruler trait - conqueror
                ruler_trait = cruel                     # Has ruler trait - cruel
                admiral_trait = buccaneer               # Has admiral trait - buccaneer
                general_trait = master_of_arms          # Has general trait - master_of_arms
                cabinet_trait = influential             # Has cabinet trait - influential
                health_trait = blind                    # Has health trait - blind
                religious_figure_trait = efficient_administrator# Has religious figure trait -
efficient_administrator
                child_trait = gifted                    # Has childhood trait - gifted
                tag = SWE                               # Belongs to Sweden
                estate = nobles_estate                  # Estate this character belongs to
                dynasty = example_dynasty               # Belongs to example_dynasty (Necessary for
noble characters)

                father = example_character_father       # example_character_father is the father of
this one
                mother = example_character_mother       # example_character_father is the mother of
this one
                spouse = example_character_spouse       # example_character_spouse is married to this
character - only one side needs this
                has_patronym = yes                      # Does the character have a patronym? False by
default
                timed_modifier = {                      # Timed Modifier notation to give characters
modifiers
                        ...
                }
                fertility = 100                         # Fertility rate




                # If we want to make our character an artist:
                artist_skill = 0.85             # Artist skill
                artist = writer                 # Artist type
                artist_trait = adept            # Artist Trait
        }

        …
}
```

Location setup

`locations` is used to define location population, starting institution spread, starting location rank setup and building templates. `define_pop` can be used to add a pop to a location, `rank` is used to set the location rank (if different than lowest rank), `town_setup` uses a building template that can be defined in `common/town_setups`. Additionally, using an institution key paired with a `= yes` can be used to make the institution present in the location at game stat. Some things regarding locations are set up outside setup, for instance raw materials are declared in location_templates.

Pop definitions

Pop definitions are added to locations according to their `size`, `type`, `culture` and `religion`. Size of 1 represents 1000 people. Example of a pop definition:

```
locations = {
        stockholm = {
                define_pop = {
                        culture = swedish       # culture of the pop
                        religion = catholic     # religion of the pop
                        type = nobles           # pop type of the pop
                        size = 1                # amount of people in thousands (0.001 = 1 person)
```

Starting population will also be impacted by several other factors.

Starting pop logic

Starting pop numbers defined by `define_pop` may differ greatly from what is available in in-game. To test pure population numbers from pop definition, the `-leavepops` commandline option, which will disable any and all calculations that might change the starting population numbers and fractions.

Here is the following factors that might impact a location's population:

- Relevant pop type will be added to fill up foreign buildings placed in others' lands.
- Extra pops will be added to fill up the pop type caps (if the location starts with 8 noble pops but the pop cap is 100, 92 will be added.) Some modifiers may not be taken into account, so the pops might still be below the theoretical maximum. As the other pop types' caps are derived from buildings, RGOs and country modifiers - those exact causes could increase the population.
- If the starting pop of a certain type is above its theoretical cap, it will be moved to peasants adequately.
- Moreover, a monthly tick is executed, so populations may differ based on starting monthly birth rate and promotion rate.
- Another thing that might impact starting population is being at war and levies being raised at game start by the game.

Starting location rank

Starting location rank is done via `rank` token. If no such rank is declared, the game will default to the lowest location rank in the hierarchy. Example:

```
locations = {
	stockholm = {
		rank = town
	}
}
```

Building templates

While individual buildings can be assigned with plenty of customization using building manager, `locations` also allows to add buildings to locations with reusable templates using `town_setup` Example:

```
locations = {
	stockholm = {
		town_setup = important_scandinavian_town
	}
}
```

wn setups

```
important_scandinavian_town = { #town setup key
        brewery = 1               # <building_type> = <amount of building levels>
        temple = 1
        tools_guild = 1
}
```

Institutions

Institutions can be added to locations by simply providing writing `<institution key> = yes` Example:

```
locations = {
        stockholm = {
                feudalism = yes
        }
}
```

Locations modifiers

Locations can be given modifiers with the timed modifiers. Example:

```
locations = {
        stockholm = {
                timed_modifiers = {
                        timed_modifiers = {
                                {
                                        modifier = <modifier_key>
                                        ...
                                }
                                {
                                        ...
                                }
                        }
                }
        }
}
```

Road network

`road_network` is used to define road connections at game start, which will by default pick the earliest road possible. Pairs of location keys are accepted here. The locations provided need to be adjacent, so a road chain must be declared location pair by location pair:

```
road_network = {
         london = barking #Create road from london location to barking location
}
```

Country setup

`countries` is used to set up data for various countries - owned locations, starting cores, discovered locations, reforms. Non-content related parts of country setup are done in country definitions. The syntax looks as follows:

```
                }
        }
}
```

Location ownership

Country entry is where the location ownership is set. There is a multitude of ways to set locations as being owned by the country and they follow the same format:

```
SWE = {
        own_control_core = {
                stockholm nykoping
        }
}
```

Here is a list of what they are and what they do:

Field Name	Integration Level Set	Does it set control?	Does it add ownership?
own_control_core	core	Yes	Yes
own_control_integrated	integrated	Yes	Yes
own_control_conquered	conquered	Yes	Yes
own_control_colony	colony	Yes	Yes
own_core	integrated	No	Yes
own_conquered	conquered	No	Yes
own_integrated	integrated	No	Yes
own_colony	colony	No	Yes
control_core	core	Yes	No
control	none	Yes	No
our_cores_conquered_by_others	core	No	No

If you are creating a non-playable pops based country, use the following format to set locations:

```
NUE = {
        type = pop
        add_pops_from_locations = {
                matootonha pabaska ruhptare marapa pocasse
        }
```

It is also possible to create a settled nation that includes the pops of unsettled regions along with settled.

```
 = {
        own_control_core = {
                guachichil
        }
```

Pops added in unsettled regions are of somewhat limited utility aside from easier settling as part of "Settle the Frontier" and other colonising actions, however, for creating semi-accurate portrayals mixed settled/tribal nations in the Americas, Siberia, and Africa this is of some use.

Basic country setup

The capital of the country is set using `capital` entry:

```
SWE = {
	capital = stockholm
}
```

One may add one or more dynasties to a country using the `dynasty` key:

```
SWE = {
	dynasty = bjalbo_atten_dynasty
	dynasty = aspenas_atten_dynasty
}
```

One can assign the country rank with `country_rank`:

```
SWE = {
	country_rank = rank_kingdom
}
```

One may also set the type of the country using `type`:

```
BRD = {
	type = building
}
```

The default is location, but it may be set as `pop`, `building`, `army` or `navy`.

Discovered locations

You can set locations as discovered using the `discovered_provinces`, `discovered_areas` and `discovered_regions` entries. One cannot discover individual locations.

```
SWE = {
	discovered_provinces = {
		gardr_province
	}
	discovered_areas = {
		celtic_sea_area
	}
	discovered_regions = {
		north_german_region
	}
}
```

ting technology level

```
SWE = {
	starting_technology_level = 3 # will research every advance with starting_technology_level <= 3
}
```

Valid for release

The `is_valid_for_release` field can be used to prevent the country from being releasable. In base game, it is only used for the Papal State. The default value, if not provided, is `yes`.

Research preferences

`ai_advance_preference_tags` can be used to make AIs more likely to research advances with certain `ai_preference_tags`.

```
SWE = {
	ai_advance_preference_tags = {
		exploration = 5 # Swedish AI will prefer to pick exploration tag by a factor of 5
	}
}
```

Culture and religion

`culture` and `religion` keys can be used to set the starting culture and religion of the country.

```
SWE = {
	culture = swedish
	religion = catholic
}
```

If those are not provided, the ones from the country definition will be used.

Additionally, in order to start with accepted and promoted cultures, the `accepted_cultures` and `tolerated_cultures` entries can be used. A culture must exist in only one of those at a time.

```
SWE = {
	accepted_cultures = { finnish gutnish }
	tolerated_cultures = { tavastian karelian kvens }
}
```

Setting starting currencies

`currency_data` can be used to set the starting amount of various currencies. It must be said that this is not the only way the starting currency data is impacted and there are other sources as well.

```
SWE = {
	currency_data = {
		gold = 10000
		prestige = 50
```

Government

government is used to set data for government related aspects:

```
SWE = {
	government = {
		ruler = random 					# the game will randomize the ruler
		# ruler = boh_charles_iv_luxembourg		# you can also set the ruler by their script
name
		consort = arb_constanza_saluzzo			# The consort, cannot accept random
		heir = ver_alberto_ii_scala 				# The heir, cannot accept random
		designated_heir_reason = infant_emperor		# Reason for the heir being heir
		active_regent = jap_oyama_hidetomo_spouse	# Active Regent

		type = republic 			# government type
		heir_selection = oligarchic_elective	# starting heir selection


		#<societal_value_type> = <value>
		centralization_vs_decentralization = 100
		traditionalist_vs_innovative = -70


		reforms = {				# List of reform keys
			veche_republic
			merchant_republic
		}


		parliament = {
			parliament_type = council	# key of the parliament type
		}

		privilege = {
			novgorod_ivans_hundred
			tysiatskii_privilege
		}

		laws = {			# all policies set at start
			#<law> = <policy set for that law>
			censorship = limited_censorship
			republican_foundation_law = republicanism_policy
		}
	}
}
```

Ruler terms

Government is also used to define who ruled the country before the start of the game. It is used for regnal history and to decide regnal numbers (Otto IV instead of Otto I):

```
ZAN= {
	government = {
		ruler_term = { character = zan_al_hassan_ibn_talut_mahdali start_date = 1277.1.1 end_date =
1294.1.1 regnal_number = 1 }
		ruler_term = { character = zan_suleiman_ibn_hassan_mahdali start_date = 1294.1.1 end_date =
1308.1.1 regnal_number = 1 }
		regnal_numbers = {
			name_chungnyeol = 1
			name_chungseon = 1
			Chungsuk = 2
		}


		inherit_ruler_terms = YMT		# A line like this can be used to inherit regnal data
	n another tag
```

Include

The `include` key can be used into include a template inside the definition. For more information see Templates.

Templates

When creating countries, one may provide a template using `include = <template_key>` statement. Templates are stored in `setup/templates` folder and include fields used in the country setup that can be reused. Unlike many other objects, the template key is represented by the filename. Example of the `subsaharan_muslim_tribe` template from `subsaharan_muslim_tribe.txt`:

```
include = "subsaharan_tribe"    # includes the subsaharan_tribe template inside.

government = {
        laws = {
                marriage_law = muslim_marriage
                heir_religion_law = heir_same_religion
                legal_code_law = sharia_law_policy
        }
}
```

Templates are only used in country setup.

Work of art manager

`work_of_art_manager` is used to setup works of art at the start of the game.

An example that illustrates what is possible:

```
work_of_art_manager = {
        painting = {    # Work of art key
                location = london       # key of the location where the WoA is at game start
                origin = canterbury     # location of where the WoA is considered to have been created
                quality = 75            # quality of the WoA, on scale from 0 to 100
                key = loc_key           # localisation key for the WoA
                creation_date = 1330.6.1# date the WoA was created
                artist = character_key  # key of the character who created this WoA (not necessary)
        }
}
```

Diplomacy manager

`diplomacy_manager` is used to set up starting rivalries, opinions and relationships between countries.

```
diplomacy_manager = {
        opinion = {
                first = SKE                     # Scanian Opinion of
                second = SWE                    # Sweden
                type = opinion_bad_monarch      # … is affected by the opinion_bad_monarch bias
        }
        trust = {
                first = SKE                     # Scanian trust of
```

```
        rival = {
                first = TEU     # Teutons consider…
                second = POL    # … Poland to be a rival.
        }


        # Diplomacy Manager can also be used to add entries from scriptable relations
        scripted_oneway = {             # if scripted relation is oneway, if it is mutual, scripted_mutual is
used
                first = ENG             # England has…
                second = SBL            # … Balliol…
                type = guarantee        # Guaranteed.
        }


        # Diplomacy Manager can also be used to define subjects
        dependency = {
                first = ENG                     # English has a subject:
                second = WLS                    # Wales
                subject_type = dominion         # of subject type “dominion”
                start_date = 1283.1.1           # since 1283.1.1
        }
}
```

Building manager

`building_manager` is used to have certain buildings present in locations at game start. For a more templated approach, see town setups

```
building_manager = {
        order_stronghold = {            #A building of type “order_stronghold”
                location = calatrava    # is present in calatrava location
                tag = CAS               # owned by CAS
                level = 1               # and is of level 1.
        }
}
```

Development setup

`development` is used to determine locations' starting development. It uses a combination of factors which add onto each other.

Development adds provinces based on:

- Being a certain location
- Belonging to a `province_definition`
- Belonging to an `area`
- Belonging to a `region`
- Having a certain `location_rank`
- Having a certain `vegetation`
- Having a certain `topography`
- Having a certain `climate`
- Being coastal
- Having a river
- Having any road connection.

```
        coastal = 1             # all coastal locations will get +1 development
        river = 2               # all locations with a river get +2 development per river size level
        road = 3                # all locations with a road get +3 development


        grasslands = -1         # all locations with this vegetation will get -1 development
        tropical = -2           # all locations with this climate will get -2 development
        flatland = -3           # all locations with this topography will get -3 development


        city = 0.5              # all locations with this location rank will get +0.5 development


        scandinavian_region = 0.25      # all locations in this region will get +0.25 development
        svealand_area = 0.25            # all locations in this area will get +0.25 development
        uppland_province = 0.25         # all locations in this province_definition will get +0.25 development
        stockholm = 0.25                # This location will get +0.25 development
}
```

International organization manager

`international_organization_manager` is used to set up international organizations at game start.

```
international_organization_manager = {
        add_international_organization = {
                type = io_key                   # what IO it is, e.g "hre" or "swiss_confederation"
                creation_date = 962.2.2         # date of creation


                map_color = rgb { 255 0 0 }     # Color used by the IO
                members = { FRA ENG }           # List of all tags that belong to the IO at game start
                leader = FRA                    # Leader of the IO
                                                # If Leader is omitted or not part of members, the first
country in members will become the leader


                # For defining what locations belong to the IO
                regions = { france_region britain_region }
                areas = { wallonia_area }
                provinces = { roman_flanders_province }
                locations = { cassel }


                elector = {                     # Key of a special status inside the IO
                        FRA ENG                 # all countries that are of that special status
                }


                laws = {                # List of starting policies selected for laws inside this IO
                        only_imperial_religion_policy
                        no_monetary_contribution
                }


                icon = daoism           # Override icon key for .dds file in
INTERNATIONAL_ORGANIZATION_TYPE_ICON_PATH


                ruler_term = {          # For declaring successive ruler terms
                        character = ogk_otto_liudolfinger       # character script key
                        start_date = 962.2.2                    # start of term
                        end_date = 967.12.25                    # end of term
                        regnal_number = 1                       # regnal number of the character (Otto I)
                }


                variables = {                                   # starting count of the IO variables
                        var_1 = 20
                        var_2 = religion:catholic
```

War manager

`war_manager` is used to set up starting wars, civil wars and truces. Truces are set up with the `truce` key, and their duration can be defined in multiple ways:

```
war_manager = {
	…
	truce = {
		attacker = GRA  # Add Granada to the attacker side
		attacker = MOR  # Add Morocco to the attacker side
		defender = CAS  # Add Castile to the defender side

		end_date = 1340.4.1     # End of the truce.
	}


	truce = {
		attacker = FRA
		defender = CAS
		start_date = 1335.6.1   # Start date of the conflict which will determine the truce
		months = 60             # the end of the truce will be in 1340.6.1, 60 months away from start
date
		# If months is not provided, TRUCE_YEARS will be used instead.
	}
	…
}
```

`civil_war` and `war` can be used to set up civil wars and wars respectively. The syntax is otherwise the same:

```
war_manager = {
	…
	war = {
		war_name = {
			name = localizable_key_for_war_name
			ordinal = 1     # will be inserted under $ORDER$ in localizable_key_for_war_name
			first = {
				name = localizable_key_for_name # will be inserted under $FIRST$ in
localizable_key_for_war_name
			}
			second = {
				name = localizable_key_for_name_2 # will be inserted under $SECOND$ in
localizable_key_for_war_name


			}
		}


		start_date = 1331.5.4   # start date for the war
		action = 1336.12.2      # used to determine last action taken in the war in order to prevent
					# wars from stalling indefinitely. Should be near game start date

		take_province = {                       # internal wargoal type which are used in “type” of
wargoals
			type = crusade_conquest # wargoal type declared in common/wargoals
			casus_belli = cb_crusade        # casus belli used in the war from common/casus_belli
			location = vilnius              # target location of the war
		}


		attacker = {
			country = TEU # country on attacking side
			request = {
				reason = Instigator #localizable identifier of reason for joining the war
			}
		}
```

```
                }
        }
        …
}
```

The first nation declared as attacker/defender is considered the war leader on the respective side.

Exploration manager

`exploration_manager` is used exclusively to add exploration preferences for tags in the game with the `add_exploration_preference` field.

```
exploration_manager = {
        add_exploration_preference = {
                country = KUR           # Country that will prefer here
                modifier = 2            # Strength of the preference


                # List of areas, regions, subcontinents, continents.
                continent = north_america
                sub_continent = india
                region = brazil_region
                area = lesser_antilles_area
                # provinces, locations not supported.
        }
}
```

Disease outbreak manager

`disease_outbreak_manager` is used to add resistance to disease in certain areas, as well as to add outbreaks.

```
 disease_outbreak_manager  = {
        add_disease_resistance = {
                type = bubonic_plague   # Disease whose resistance we are adding
                resistance = 0.1                # Resistance we are setting


                # List of areas, regions, subcontinents, continents, province_definitions, locations
                locations = {
                        paris london
                }
                provinces = {
                        south_holland_province
                        north_holland_province
                }
                areas = {
                        north_portugal_area
                        south_portugal_area
                }
                regions = {
                        italy_region
                }
                sub_continents = {
                        eastern_europe
                }
                continents = {
                        asia
                }
        }
```

```
                origin = 1000                   # location id, internal ROBOT can we get key here?



                # List of areas, regions, subcontinents, continents, province_definitions, locations
                locations = {
                        paris london
                }
                provinces = {
                        south_holland_province
                        north_holland_province
                }
                areas = {
                        north_portugal_area
                        south_portugal_area
                }
                regions = {
                        italy_region
                }
                sub_continents = {
                        eastern_europe
                }
                continents = {
                        asia
                }
        }
}
```

Contrary to what the name might imply, both of the effect actually set the values, so if you were to add your own resistance to a certain disease, the existing resistance will be replaced.

Additionally; the game seems to add +5% resistance on top of what is declared.

Colony manager

`colony_manager` is used to define starting colonial treaties.

```
colony_manager = {
        vasterbotten_province = {       # Only province_definitions can be used for colonial claims
                tag = SWE               # country affected
                category = exclusive    # should SWE be the only tag allowed to colonize?
                # category = same_religion      # should nations of SWE's religion be barred from colonizing?
                reason = new_treaty     # localizable string serving as identifier for localisation.
        }
}
```

Timed modifiers

Several types in the setup managers allow for the addition of `timed_modifiers` which allow to add starting static_modifiers to various objects. Timed modifiers are usually created with the following syntax:

```
{
        modifier = "herring_market"     # Modifier name
        start_date = 1111.1.1           # When the modifier was given
        end_date = 1340.1.1             # When the modifier will expire
        size = 1                        # Size of the modifier (1 = normal size, 2 = double the modifiers, 0.5
```

Countries folder

Outside the countries, there is also a specialized folder for **country_definitions** in `setup/countries`. Files here are used to give countries colors, unit colors, default cultures and religions and regnal names.

All entries used in Country setup must also be created here.

Example of an entry:

```
TUR = {
        color = map_ottomans            # primary color - used for the mapmode
        color2 = rgb { 126 203 120 }    # secondary color
        color3 = hsv { 100 100 100 }    # tertiary color

        # Unit colors - used for units on the map
        unit_color0 = rgb {     175 40 40 }
        unit_color1 = rgb {     76 78 114 }
        unit_color2 = rgb {     18 90 47 }

        # Culture and religion definitions - used to determine the culture and religion of this tag when
released
        culture_definition = turkish_culture    # Culture definition
        religion_definition = sunni             # Religion definition.

        male_regnal_names = { name_key_1 name_key_2 }   # Special male regnal names unique to this tag
(localizable key list)
        female_regnal_names = { name_key_3 name_key_4 } # Special female regnal names unique to this tag
(localizable key list)

        is_historic = yes               # Field to mark a country as "historic" -> it is meant to not exist
nor to have any cores. It is meant to be used as a "country that once existed" in GUIs

        description_category = military # is the country considered a) administrative (Economy) b) diplomatic
(Politics) c) military (Expansion)
        difficulty = 1                  # Difficulty (from 1 (very easy) to 5 (very hard) -> this value is
shown in Country Selection in the lobby
}
```

References

Modding Return to top

Documentation	Defines • Effects • Scopes • Scope links • Triggers | Colors • Macros • Mean time to happen • Modifier types • On actions • Script value • Variables | GUI script • Localization
Scripted content	Actions • Disasters • Events • Missions • Modifiers • Scripted gui • Setup • Situations • Customizable localization
Scripted types	Advances • Art • Buildings • Bureaucracies • Casus belli • Characters • Concepts • Countries • Culture • Diplomacy • Diseases • Estates • Goods • Institutions • International organizations • Laws • Movements • Peace treaties • Pops • Religion • Subject types • Traits • Units • Wargoals
Map	Map • Map modes • Terrain
Graphics	3D Models • Interface • Graphical assets • Fonts • Flags
Audio	Music • Sound
Other	AI • Console commands • Checksum • Mods • Mod compatibility • Mod structure • Troubleshooting
Guides	Interface modding guide • Mod translation • Save-game editing • Settlement position modding guide
Tools	Arcanum • PDX DeepL • PDX Workshop Manager • Community Mod Toolkit • **Add Your Tool to the Wiki**

Paradox Wikis

Ara

GAMES

Discover

Our Brands

Browse

Play on Paradox technology

ABOUT

News

About us

Careers

Join our playtests

Media contact

COMMUNİTY

Paradox Forums

Paradox Wikis

Support

SOCİAL MEDİA